Econometrics

Dr. Mutua Samuel

`Week 4 Notes`

Contents

1 Time Series

1.1 Introduction

A time series is ant series of data that varies over time.

Examples include:

- Payroll employmnet in Kenya
- Unemployment rates
- Daily prices of stocks and shares
- Quartely GDP series

1.2 Stationarity

1.2.1 Weak Stationarity

A time series is weakly stationarity if its mean and variance are constant over time and value of covrainace between the priods depend on the lags between the two periods.

- Mean $E(Y_t) = \mu$
- $Var(Y_t) = E[Y_t - \mu]^2 = \sigma^2$
- Covariance $\rho_k = E[(Y_t - \mu)(Y_{t-k} - \mu)^2]$

1.2.2 Strong Stationarity

A time series is strongly stationary if for any values $j_1, j_2, ..., j_n$ the joint distribution of $(Y_t, Y_{t+j_1}, ..., Y_{t+j_n})$ depends only on the lag and not the time itself.

To test for stationarity we use the Augmented Dickey Fuller test and the hypothesis is:

H_0: Time series is non-stationary H_a: Time series is stationary

We use the `adf.test()` method from the tseries library

```
google_2015 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2015)
adf.test(google_2015$Adj_Close)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  google_2015$Adj_Close
## Dickey-Fuller = -2.1101, Lag order = 6, p-value = 0.5295
## alternative hypothesis: stationary
```

If the time series is not stationary, we difference the series to make the data stationary.

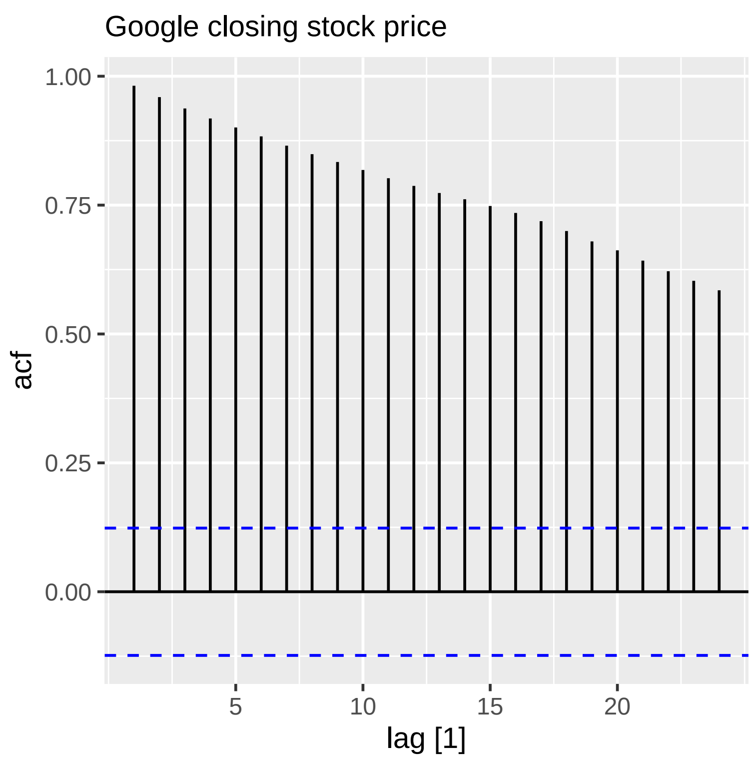
1.3 Differencing

One way to make non-stationary time series stationary is to compute the differences between consecutive observations.

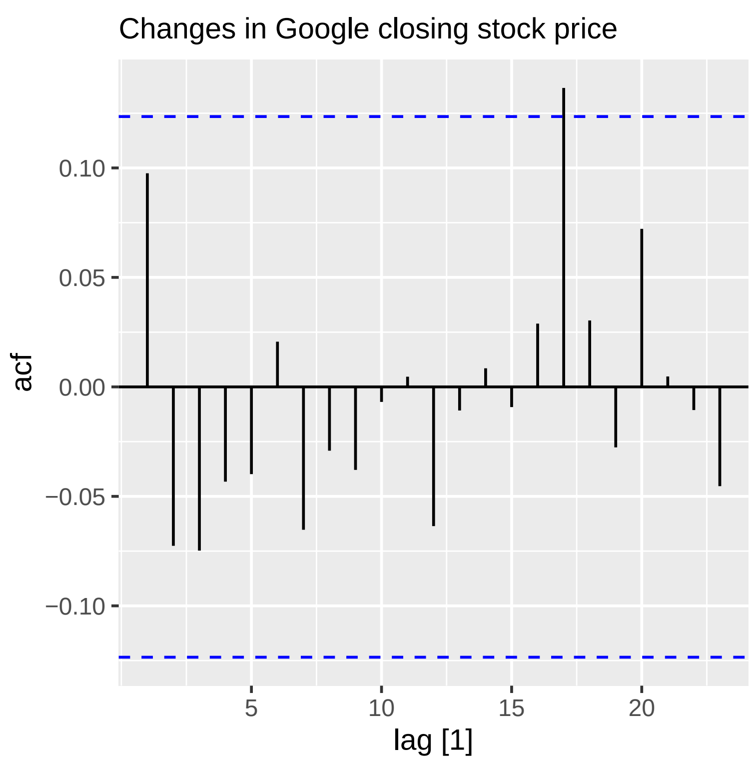
The ACF plot is also useful for identifying non-stationary time series.

For a stationary time series, the ACF will drop to zero relatively quickly while the non-stationary data decreases slowly.

```
google_2015 |> ACF(Close) |>
  autoplot() + labs(subtitle = "Google closing stock price")
```




```r
google_2015 |> ACF(difference(Close)) |>
  autoplot() + labs(subtitle = "Changes in Google closing stock price")
```



We can also use Ljung-Box test to check for independence in the data hence stationary.

H_0 : Stationary data H_a : Non-stationary data

```
google_2015 |>
  mutate(diff_close = difference(Close)) |>
  features(diff_close, ljung_box, lag = 10)
```

```
## # A tibble: 1 x 3
##   Symbol lb_stat lb_pvalue
##   <chr>    <dbl>     <dbl>
## 1 GOOG      7.91     0.637
```

1.4 Backshift Operator

The backward shift operator is useful when working with time series lags:

$$BY_t = Y_{t-1}$$

Two applications of B to Y_t shifts the data back two periods:

$$B(BY_t) = B^2Y_t = Y_{t-2}$$

The backward shift operator is useful for describing the process of differencing. A first difference can be written as:

$$Y'_t = Y_t - Y_{t-1} = Y_t - BY_t = (1 - B)Y_t$$

2 Autoregressive Models

The term autoregression indicates that it is a regression of the variable against itself.

Thus an autoregressive model of order p can be written as:

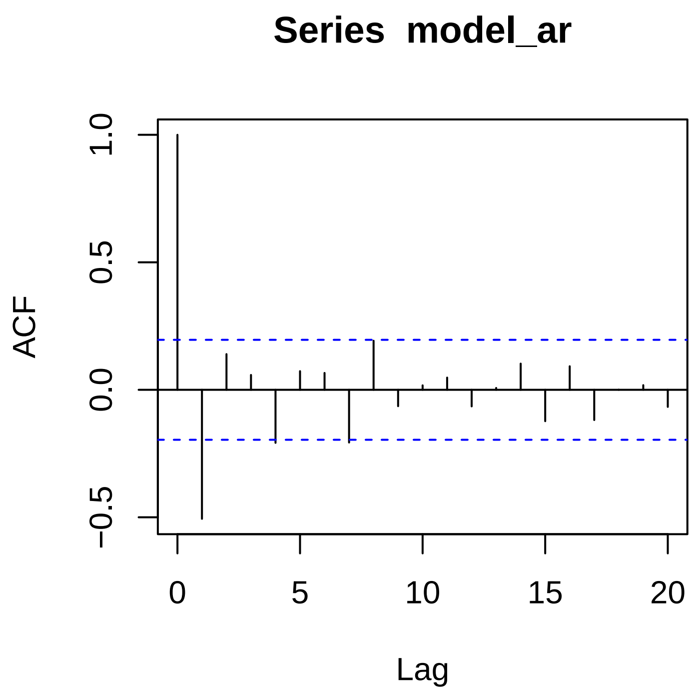
$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + ... + \phi_p Y_{t-p} + \epsilon_t$$

This is like a multiple regression model but with lagged values of Y_t as predictors.

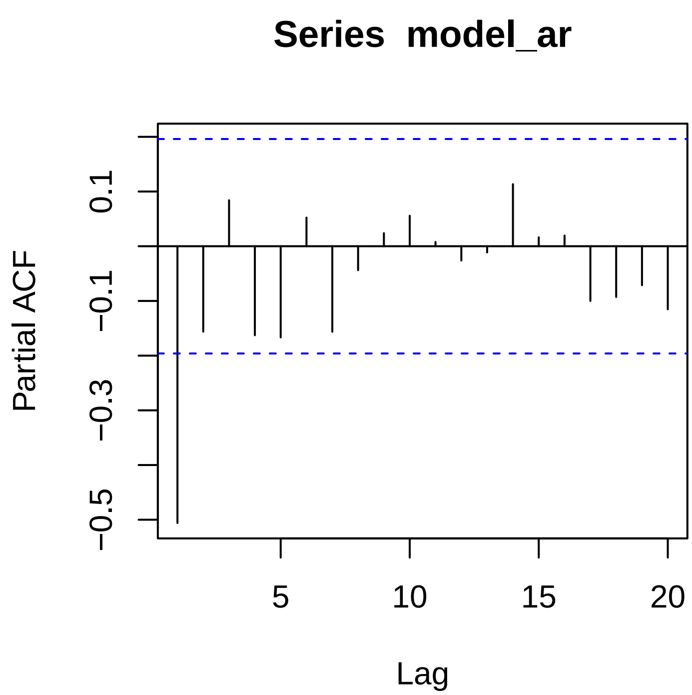
We normally restrict autoregressive regression models to stationary data, in which some constraints on the values of the parameter is required.

We can simulate the AR model in R as:

```
model_ar <- arima.sim(list(order = c(1,0,0), ar = -0.5), n = 100)
acf(model_ar)
```




```
pacf(model_ar)
```



The ACF decays exponentially and the PACF cuts off at lag p.

3 Moving Average Process

Rather than using past values of the forecast variable in the regression, a moving average model uses past forecast errors in a regression like model.

$$Y_t = c + \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2} + ... + \theta_q\epsilon_{t-q}$$

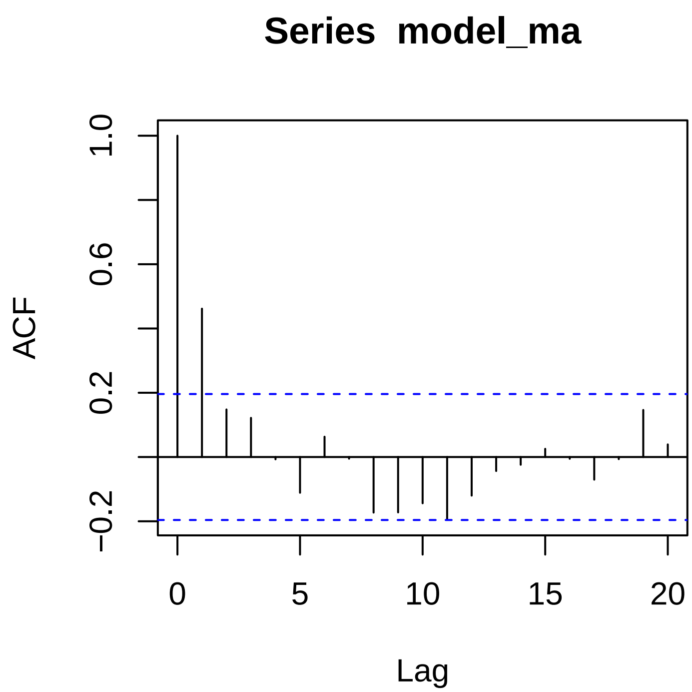
We refer to this as an MA(q) model.

It is possible to write any stationary AR(p) model as an $MA(\infty)$.

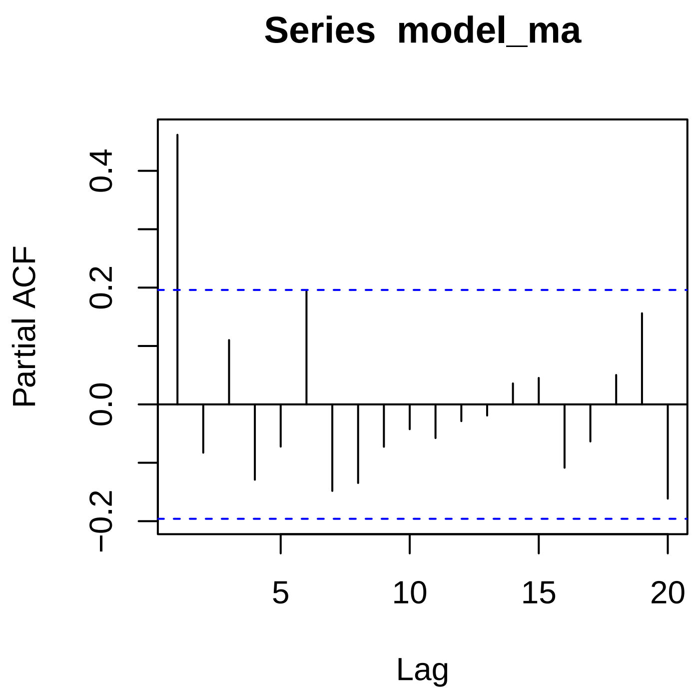
MA model is called invertible that is we can write any invertible MA(q) process as an $AR(\infty)$ process.

We can simulate an MA(q) process in R using:

```
model_ma <- arima.sim(list(order = c(0,0,1), ma = 0.5), n = 100)
acf(model_ma)
```




```
pacf(model_ma)
```



4 ARIMA Model

If we assume that the series is partially autoregressive and partially MA we obtain the following time series:

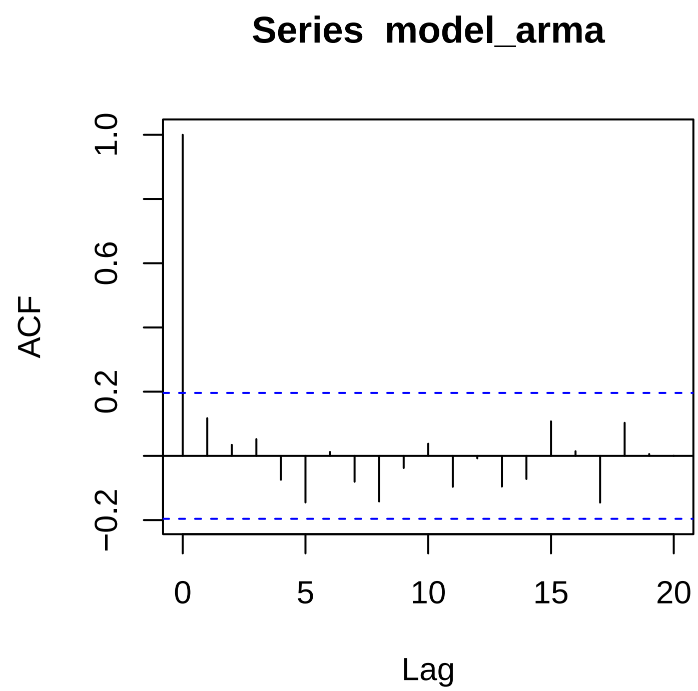
$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + ... + \phi_p Y_{t-p} + e_t - \theta_1 e_{t-1} - ... - \theta_q e_{t-q}$$

It is called an ARMA(p,q)

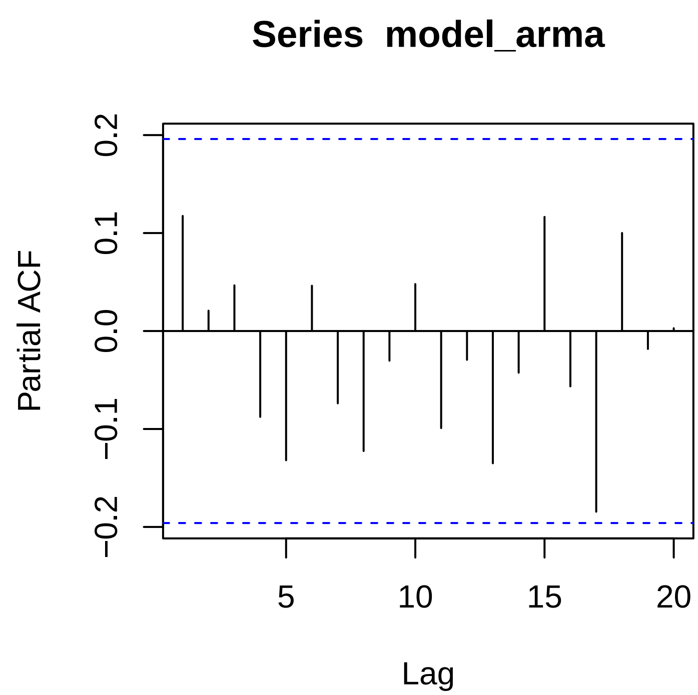
The ACF of ARMA behaves like the AR and the PACF behaves like the MA

We can simulate the ARMA models as follows:

```
model_arma <- arima.sim(list(order = c(1,0,1), ma = 0.5, ar = -0.5), n = 100)
acf(model_arma)
```




```r
pacf(model_arma)
```



4.1 ARIMA Modeling in `fable`

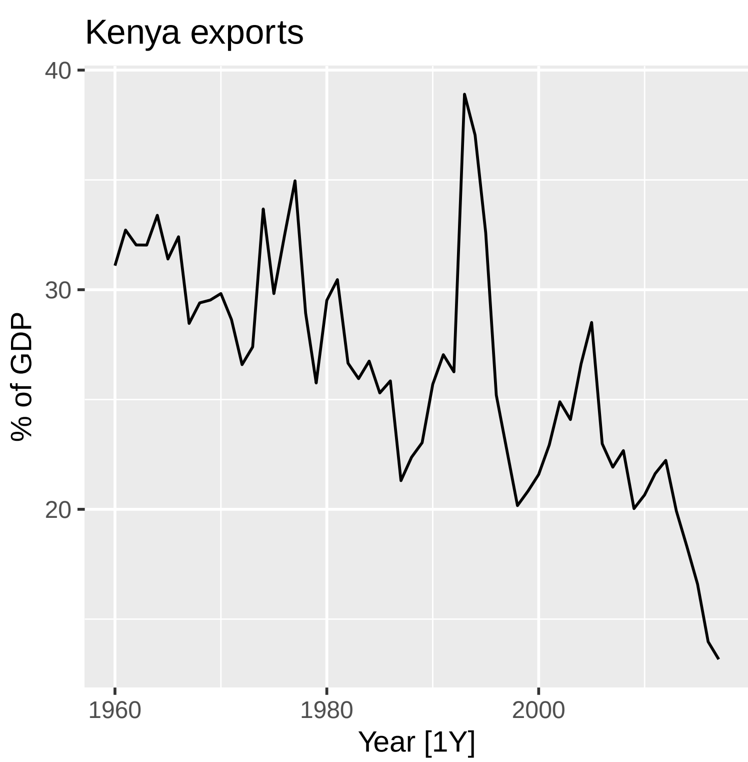
When fitting an ARIMA model to a set of non-seasonal data the following procedure provides a general approach:

- Plot the data and identify any unusual observations
- If necessary transform the data to stabilise the variance
- If the data is non-stationary take the first difference until the data are stationary
- Examine the PACF/ ACF
- Try the chosen model and use the AIC to search for optimal model
- Check the residuals from your chosen model by plotting the ACF of the residuals, and doing a portmanteau test of the residuals. If they do not look like white noise, try a modified model.
- Once the residuals look like white noise, calculate forecasts.

4.1.1 Example: CAR Exports

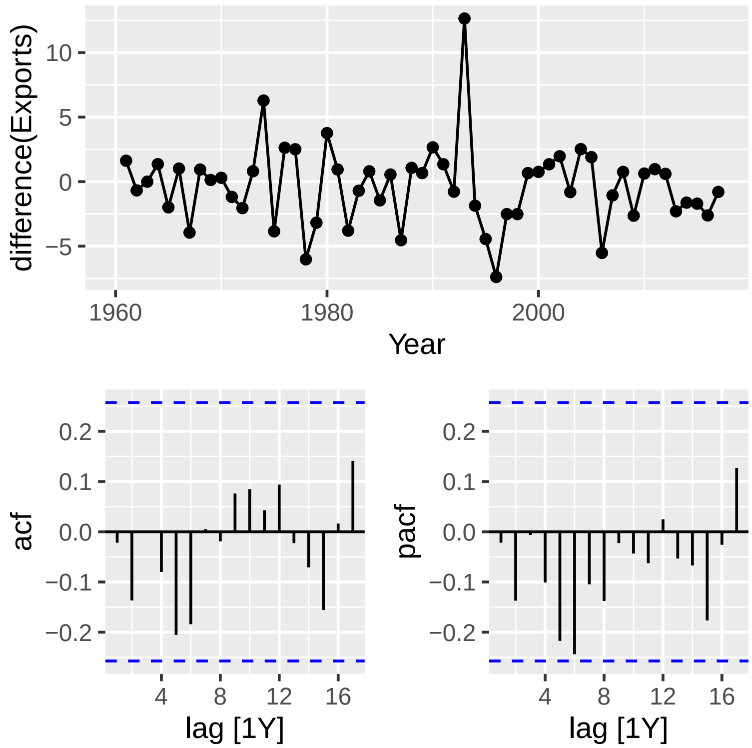
We demonstrate the procedure listed here in the above data.

```
global_economy |>
  filter(Code == "KEN") |>
  autoplot(Exports) +
  labs(title="Kenya exports",
       y="% of GDP")
```



We address the non-stationarity of the data by taking the difference

```
global_economy |>
  filter(Code == "KEN") |>
  gg_tsdisplay(difference(Exports), plot_type='partial')
```



We fit the models as shown

```
caf_fit <- global_economy |>
  filter(Code == "KEN") |>
  model(arima210 = ARIMA(Exports ~ pdq(2,1,0)),
        arima013 = ARIMA(Exports ~ pdq(0,1,3)),
        stepwise = ARIMA(Exports))
caf_fit |> pivot_longer(!Country, names_to = "Model name",
                         values_to = "Orders")
```

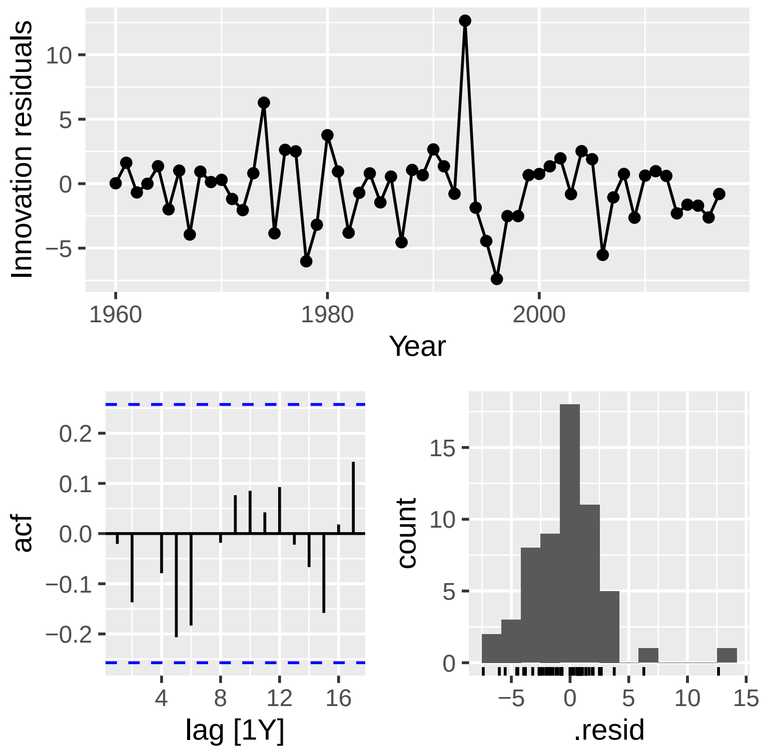
```
## # A mable: 3 x 3
## # Key:     Country, Model name [3]
##   Country `Model name`         Orders
##   <fct>   <chr>               <model>
## 1 Kenya   arima210     <ARIMA(2,1,0)>
## 2 Kenya   arima013     <ARIMA(0,1,3)>
## 3 Kenya   stepwise     <ARIMA(0,1,0)>
```

```
glance(caf_fit) |> arrange(AICc) |> select(.model:BIC)
```

```
## # A tibble: 3 x 6
##   .model   sigma2 log_lik   AIC  AICc   BIC
##   <chr>     <dbl>   <dbl> <dbl> <dbl> <dbl>
## 1 stepwise   9.45   -145.  292.  292.  294.
## 2 arima210   9.63   -144.  295.  295.  301.
## 3 arima013   9.57   -144.  296.  296.  304.
```

We check the residuals of the model

```
caf_fit |>
  select(stepwise) |>
  gg_tsresiduals()
```



The ACF shows that all autocorrelations are within the threshold limits, indicating that the residuals are behaving like white noise.

We test for adequacy of the model

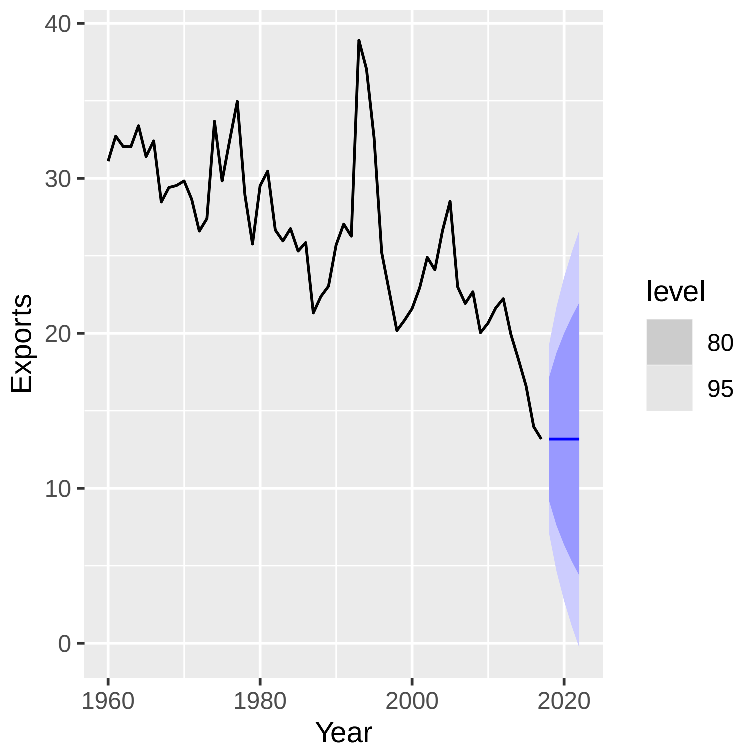
```
augment(caf_fit) |>
  filter(.model=='stepwise') |>
  features(.innov, ljung_box, lag = 10, dof = 3)
```

```
## # A tibble: 1 x 4
##   Country .model   lb_stat lb_pvalue
##   <fct>   <chr>      <dbl>     <dbl>
## 1 Kenya   stepwise    7.60     0.369
```

The p-value is large enough suggesting that the residuals are white noise.

We display the forecasts for the chosen model

```
caf_fit |>
  forecast(h=5) |>
  filter(.model=='stepwise') |>
  autoplot(global_economy)
```


Exports
Year
level
80
95
0
10
20
30
40
1960
1980
2000
2020